

Yep! 😊 These are **two separate interview questions**, but they connect nicely.

1. Page Router vs App Router — why did Next.js have two?

The simple interviewer answer:

Next.js originally had the Pages Router. Later, Next.js introduced the App Router to provide a newer architecture built around React Server Components, nested layouts, streaming, and more granular control over rendering and data fetching. The Pages Router was kept for backward compatibility, so both routing systems exist.

That's the clean answer. 🎯

Pages Router

Introduced first:

```
pages/  
├─ index.tsx  
├─ about.tsx  
└─ products/  
    └─ [id].tsx
```



Routing is based on the `pages/` directory.

It commonly uses:

```
getServerSideProps()  
getStaticProps()  
getStaticPaths()
```



Mental model:

```
pages/
  ↓
Pages Router
  ↓
traditional SSR / SSG / ISR model
```



App Router

Newer architecture:

```
app/
├─ page.tsx
├─ layout.tsx
├─ products/
│   └─ [id]/
│       └─ page.tsx
```



It introduced things such as:

```
React Server Components
nested layouts
loading.tsx
error.tsx
Suspense / streaming
Server Actions
new caching/data-fetching model
```



Mental model:

```
app/
  ↓
App Router
  ↓
React Server Components
  ↓
more granular rendering + caching
```



Why didn't Next.js just replace Pages Router?

Because there are **millions of existing Next.js applications** using:

```
pages/
```



If Next.js suddenly removed it, existing applications would break.

So:



And importantly, a project can even have **both**:

pages/
app/

during a gradual migration.

🎤 Interview version

If they ask:

"Why are there two routing systems in Next.js?"

Say:

"Next.js started with the Pages Router. The App Router was later introduced as a new architecture around React Server Components, nested layouts, streaming, and newer server-side capabilities. The Pages Router remains supported for backward compatibility, so existing applications don't have to migrate all at once."

That's plenty. Don't give them a 15-minute history lecture. 😂

2. What's a Network Edge?

This one sounds fancy but is actually simple.

Imagine your Next.js server is physically located in:

🇺🇸 USA

and you're a user in:

 India

Without an edge/CDN:

 India
↓
Internet
↓
USA server
↓
response
↓
 India

That's potentially a long network journey.

With a CDN / Edge

A CDN has servers distributed around the world:

 CDN / Edge Network India
|
 Singapore
|
 Germany
|
 USA

The **edge** means a server/location that's geographically closer to the user, near the **edge of the network**.

So:

User in India
↓
Nearby edge server
↓
cached response



Instead of:

India → USA → India



you may get:

India → nearby edge → India



Much faster. 🚀

🧠 Why does this matter for our cache discussion?

Remember:

Browser



CDN / HTTP Cache



Next.js



That CDN is typically made up of **edge locations**.

Suppose `/products` has a cacheable HTTP response.

A user in India requests it:

🇮🇳 User



🇮🇳 Edge



💾 cached HTTP response



The request may **never reach your Next.js application server**.

That's one of the huge benefits of edge caching.

⚡ Edge vs Next.js Data Cache

Don't confuse these.

Edge/CDN cache

Caches something like:

HTTP response



HTML / response body / headers



and lives **outside your application**.

Next.js Data Cache

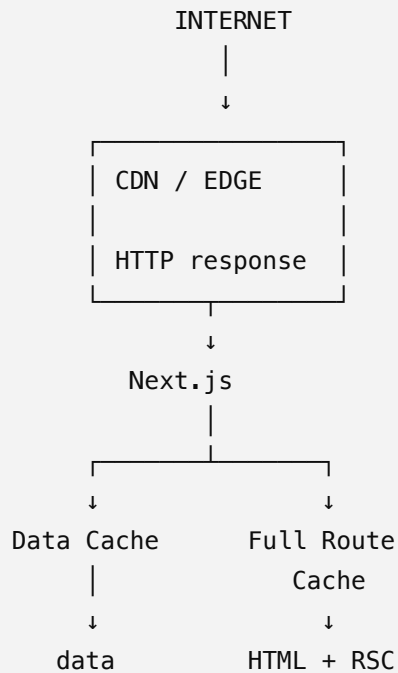
Caches something like:

```
fetch()
↓
API data
```



and is part of the Next.js/server caching architecture.

So:



And then:

```
Next.js
↓
RSC Payload
↓
Browser
↓
Router Cache
```



Now your four cache layers have a very clean separation.

Interview cheat sheet

If they ask:

Pages Router vs App Router?

Pages Router is the original `pages/` routing architecture. App Router is the newer `app/` architecture built around React Server Components, nested layouts, streaming, and newer server-side capabilities. Both exist because the Pages Router remains supported for backward compatibility.

What's the network edge?

The network edge refers to servers geographically distributed closer to users, typically through a CDN. Cacheable responses can be served from an edge location instead of traveling all the way to the origin application server.

And if they ask "origin server?":

The origin is the server that is ultimately responsible for generating or supplying the response when the edge doesn't already have a valid cached response.

So:

```
User
  ↓
Edge / CDN
  ↓ cache miss
Origin / Next.js
  ↓
Response
  ↓
Edge
  ↓
User
```



That's the whole thing. 🤓🔥



what exactly mean by backward compatability? CDN vs edge network?

Yep 🤓 — these are two terms worth getting **really crisp** for interviews.

1 What does "backward compatibility" mean?

It means:

A newer version continues to support things that were built using the older

version.

Simple example

Imagine a company has a Next.js application built in 2022:

```
my-app/  
└─ pages/  
    ├── index.tsx  
    └─ about.tsx
```



Then Next.js introduces the App Router:

```
my-app/  
└─ app/  
    ├── page.tsx  
    └─ about/  
        └─ page.tsx
```



If the new Next.js version completely removed support for:

```
pages/
```



then thousands/millions of existing applications would need to be rewritten immediately.

Instead:

```
Old application  
pages/  
  ↓  
still works  
  ↓  
New Next.js version
```



That's **backward compatibility**.

Interview answer

"Backward compatibility means newer versions continue to support older APIs or architectures so existing applications don't suddenly break when they upgrade."

2 CDN vs Edge Network

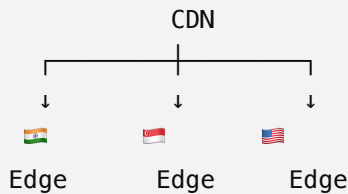
This is a little subtle because people often use these terms almost interchangeably.

CDN

CDN = Content Delivery Network.

It's a **service/network** designed to **distribute and deliver content closer to users.**

For example:



The CDN has multiple locations around the world.

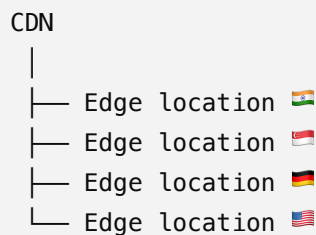
Edge network

The **edge network** is the distributed network of locations/servers that sit closer to users.

Those individual locations are often called:

edge locations
edge servers
edge nodes

So you can think:



Analogy

Think of a pizza company 🍕

CDN = the entire delivery network

Pizza Company
↓
Delivery network

Edge location = one nearby branch

Pizza Company
↓
Hyderabad branch
↓
Your house



Instead of:

California restaurant
↓
India
↓
Your house



you have:

Nearby edge location
↓
Your house



But there's an important distinction

A CDN is not just "a bunch of edge servers."

The CDN is the **delivery system/service** that uses distributed infrastructure, caching, routing, etc.

The edge is the **geographically distributed execution/delivery location**.

So:

CDN
↓
distributed edge locations
↓
serve/cache content near users



3 Where does this fit with Next.js?

Imagine your Next.js app is deployed with a CDN:

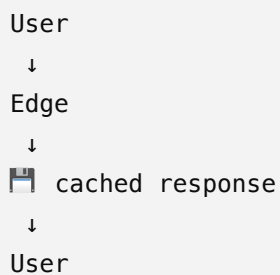


A user requests:

/products

The CDN can route them to an appropriate edge location.

If that edge already has a valid cached HTTP response:



The **origin/Next.js** server isn't contacted for that request.

If there's no cached response:



The edge may then cache the response if the relevant HTTP caching rules allow it.



One distinction you should memorize

CDN

=

the distributed content-delivery service/network



Edge

=

a location/server close to the user within that distributed network



And:

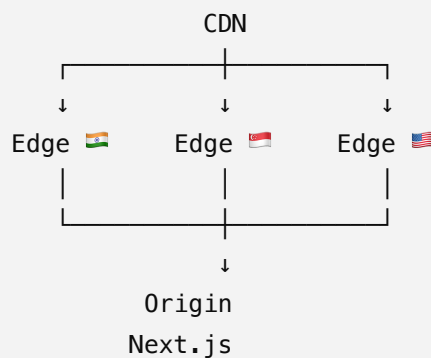
Origin

=

the source/backend that produces the response



So the simplest picture is:



Interview one-liner

"A CDN is the distributed network/service that delivers content closer to users. An edge location is one of the geographically distributed servers or locations within that network. The origin is where the content is ultimately generated or fetched from."

That's interview-ready. 🔥



Let's do it 🤔🔥 This is the piece that makes **Router Cache** finally click.

Layer 2.5 — `<Link>` Navigation vs Refresh

We'll imagine this app:

```
/products
/products/123
/cart
```



And assume `/products/123` is dynamically rendered but its product data is cached.

🟢 Step 1 — User visits `/products`

The user initially opens:

```
https://example.com/products
```



For a simplified flow:

```
Browser
  ↓
CDN / Edge
  ↓
Next.js
  ↓
render / cached route
  ↓
HTML + RSC Payload
  ↓
Browser
```



The browser receives:

```
HTML
+
RSC Payload
```



The App Router can now keep relevant RSC information in:

```
Router Cache
  ↓
/products → RSC Payload
```



So now:

SERVER

CLIENT



Data Cache



Full Route Cache



HTML + RSC



Router Cache



/products

Step 2 — User clicks `<Link href="/products/123">`

Now they click:

`</> TypeScript`

```
<Link href="/products/123">
  iPhone
</Link>
```

This is **client-side navigation**.

The browser doesn't simply behave like a traditional:

`</> HTML``</>`

```
<a href="/products/123">
```

where the browser performs a full document navigation.

Instead, Next.js can request the required **RSC payload** and update the relevant UI.

Conceptually:

Click Link



Next.js Router



request RSC payload



Next.js server



RSC Payload



Browser





Router Cache

Now:

Router Cache



/products → RSC

/products/123 → RSC

Step 3 — User clicks /cart

Again:

</> TypeScript



```
<Link href="/cart">
```

Cart

```
</Link>
```

Client-side navigation:

Browser



Next.js Router



request needed RSC



Server



RSC Payload



Router Cache



Now the client might have:

Router Cache



/products

/products/123

/cart

Step 4 — User goes back to /products/123

Now something interesting can happen.

The client already has:

```
/products/123
  ↓
RSC Payload
```



in its Router Cache.

So the client can potentially reuse that cached client-side route data rather than fetching it again.

That's one reason client-side navigation can feel **really fast**. 🚀

🔥 Now compare that with REFRESH

This is where you need to separate the concepts.

Suppose you're at:

```
/products/123
```



and press:

```
Cmd + R
```



That's a **full document refresh**.

The browser starts a new request:

```
Browser
  ↓
CDN / Edge
  ↓
Next.js
```



The existing in-memory Router Cache isn't something you should think of as surviving that full browser reload.

So:

```
Refresh
  ↓
new document request
```



↓
server/CDN involved

Whereas:

<Link>
↓
client-side navigation
↓
RSC navigation
↓
Router Cache can participate



This gives you the key difference

<Link>

Client navigation
↓
RSC payload
↓
Router Cache
↓
update UI



Browser refresh

Full document request
↓
CDN / server
↓
HTML + RSC
↓
new client session/state



But don't say "Link never hits the server"

This is a common interview mistake.

Don't say:

"<Link> means no server request."

✗ Not necessarily.

Instead say:

"`<Link>` enables client-side navigation. Next.js can request the required RSC payload from the server rather than performing a full document navigation."

That's much more accurate.

🎯 Now put all the caches into our example

Suppose:

/products/123



has:

Product data



Data Cache ✓



and the route is dynamically rendered:

Full Route Cache ✗



The user navigates with `<Link>` :

`<Link href="/products/123">`



Flow:

Browser

client navigation

Next.js Router

Server

Data Cache



HIT ✓



Dynamic render



RSC Payload



Browser



Router Cache

Notice something beautiful here:

Dynamic rendering

+

Data Cache HIT

+

Router Cache



can all coexist.



This is exactly why we kept the layers separate.



Your final mental model

When a user clicks a `<Link>`:

`<Link>`



Client Router



Need RSC data?



YES



NO



Server



Router Cache



Data Cache



render



RSC Payload



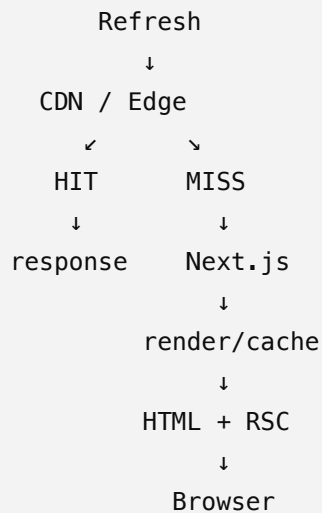
Router Cache



UI



When the user refreshes:



★ Interview-ready sentence

If they ask:

"What's the purpose of the Router Cache?"

Say:

"The Router Cache is a client-side cache used by the Next.js App Router to retain RSC payloads for visited or prefetched route segments, which helps make client-side navigation faster and lets the router preserve navigation state."

That's a very solid answer. 🤖 🔥

📄 👍 ⬆️ ↺ ...

ok

Perfect 🤖 🔥

At this point, you've got the **four cache layers + rendering decision** pretty well separated.

The next piece I'd teach is **cache invalidation**, because that's where everything becomes practical:

Data Cache

↓

How do I invalidate it?

Full Route Cache

↓

How does it get regenerated?

Router Cache

↓

How does the client get updated?

CDN

↓

How does HTTP caching/revalidation fit in?

Then we'll connect that to:

```
revalidatePath()
revalidateTag()
updateTag()
refresh()
```



and especially **Server Action vs Route Handler**, which was one of your earlier sticking points. 🔥



but we covered this topics right

Yes! 🤪🔥 **We already covered cache invalidation.** You're right.

Earlier we went through:

```
revalidateTag()
updateTag()
revalidatePath()
refresh()
```



and also discussed:

Server Action vs Route Handler



including how invalidation differs depending on where you call them.

So we shouldn't redo that. 👍

Where we are now

You've essentially covered:

- ✓ Layer 1
 - Route structure
 - Dynamic segments
 - Static vs dynamic rendering
 - Data caching
- ✓ Layer 2
 - Data Cache
 - Full Route Cache
 - Router Cache
 - CDN / HTTP Cache
 - Edge vs origin
 - TTL
 - Revalidation
 - <Link> navigation vs refresh
- ✓ Cache invalidation
 - revalidatePath
 - revalidateTag
 - updateTag
 - refresh
 - Server Action vs Route Handler



 The next NEW topic should be:

Modern Next.js caching — Cache Components

Specifically:

```
"use cache"  
  ↓  
cacheLife()  
  ↓  
cacheTag()  
  ↓  
PPR / Partial Prerendering  
  ↓  
How this differs from the older  
Data Cache / Full Route Cache model
```



This is important because we're intentionally learning **modern Next.js**, rather than accidentally drilling an older caching model into your head. 😎

So yes — **skip invalidation repetition. We move forward to modern Cache Components.** 🔥



ChatGPT can make mistakes. Check important info.